

Evolvable Representations

Learning slow features that make fast, gradient-free evolutionary adaptation provably easy

Research proposal — semester project / practical work, ETH Zürich

Tom Offermann

MSc Computer Science (Machine Intelligence, minor Theoretical Computer Science), ETH Zürich
toffermann@ethz.ch

Draft of September 21, 2026

Abstract

Deployed models should keep adapting, often from nothing more than a scalar reward. Evolutionary algorithms (EAs) are the natural tool for this regime: they need only fitness values, parallelise perfectly and handle discrete, non-differentiable decisions. They also come with runtime theory. For *additive* fitness landscapes, a (1+1) EA re-adapts after a change of k positions in $O(n \log k)$ evaluations. Whether a real adaptation problem is additive, however, is not a property of the task alone. It is a property of the *representation* on which the evolved parameters act. This proposal makes that representation the object of study. A slow, gradient-trained encoder feeds a tiny, discrete, evolved read-out (the *genome*). We ask how the encoder must be shaped so that the fitness landscape seen by the EA is nearly additive, and related tasks have nearby optima. We (i) define the *evolvability* of a representation as the expected number of reward evaluations an EA needs to move between related tasks; (ii) prove that, for threshold read-outs over binary features, epistasis is bounded by feature co-activation near the decision threshold; (iii) propose *evolution-in-the-loop* meta-training, where an inner EA solves tasks and the encoder is updated by gradients at the genomes the EA found, regularised towards low epistasis; and (iv) lay out a staged experimental plan from a synthetic task family with known ground truth to reward-only adaptation on real images, sized for CPU clusters and a single T4 GPU, with a direct scaling path to reward-only routing adaptation in mixture-of-experts language models.

The question in one sentence

Meta-learning (MAML and friends) shapes representations so that *gradient descent* adapts fast. We ask the evolutionary counterpart: **what must a learned representation look like so that a simple EA, seeing only scalar rewards, adapts fast and provably so, and can such a representation be *trained*?**

Contents

1	Motivation	3
2	Setting and notation	4
2.1	Task family, encoder, genome	4
2.2	Evolvability of a representation	4
2.3	Two knobs: ruggedness and task distance	5
3	Where epistasis comes from: a feature-level bound	6

4	Method: evolution-in-the-loop meta-training	7
4.1	Objective	7
4.2	Algorithm	7
4.3	Why this is cheap	8
5	Research questions and hypotheses	8
6	Experimental plan	8
6.1	Stage 1 — synthetic task family with known ground truth (CPU)	8
6.2	Stage 2 — images with known factors (CPU / T4)	8
6.3	Stage 3 — reward-only adaptation on natural images (T4)	9
6.4	Stage 4 — scaling path (with large compute)	9
6.5	Arms	10
6.6	Metrics	10
6.7	Compute and timeline	11
7	Related work	11
8	Expected contributions and outlook	12

1 Motivation

Adaptation after deployment, from reward alone. A model that is deployed into a changing world rarely receives labelled gradients. It receives outcomes: a task succeeded or failed, a user accepted or rejected, a robot moved forward or fell over. In this *reward-only* regime, gradient-based fine-tuning is either unavailable or requires high-variance estimators (policy gradients), and it moves all parameters at once, which causes forgetting.

Why evolutionary algorithms. EAs optimise any scalar objective, evaluate candidates in parallel, and handle discrete choices natively. Their weakness is dimension: in n dimensions, a population of size N yields a search direction whose signal fraction is only $N/(N + n + 1)$. This is why they fail on 10^9 parameters. The remedy, developed in our earlier working document, is a *slow/fast split*. A large encoder is trained by gradients and then frozen; only a tiny genome of $n \approx 10^2$ discrete values, placed at the very end, is evolved. Placing it late also makes the population cheap: the encoder runs once per input and all N genomes are evaluated on cached features.

The missing piece: the landscape is made by the representation. The runtime of an EA is governed by the *structure* of its fitness landscape. On additive (OneMax-like) landscapes, a (1+1) EA is fast and its behaviour is understood exactly. On rugged, epistatic landscapes (NK landscapes, needle-in-a-haystack), it can be exponentially slow. In the slow/fast split, the landscape over the genome is $g \mapsto F_\tau(g; \phi)$. It depends on the task τ *and on the encoder* ϕ . The same task can be trivial for an EA on one representation and hopeless on another (Figure 1). Current practice picks ϕ for reasons unrelated to evolutionary adaptation (supervised pretraining, reconstruction, contrastive objectives), and then hopes.

This proposal. We make the representation’s *evolvability*, in the sense of evolutionary biology, the capacity to generate adaptive variation, an explicit, measurable and trainable quantity:

- C1 Definition.** The evolvability of a representation is the expected number of reward evaluations an EA needs to adapt between related tasks (Definition 2.1). It decomposes into a *ruggedness* term (epistasis) and a *task-distance* term (how far apart the optima of related tasks lie).
- C2 Theory.** For threshold read-outs over binary features, pairwise epistasis is bounded by the probability that both features are co-active *and* the example lies near the decision threshold (Lemma 3.1). This turns a landscape property into a property of the features that can be regularised directly. Combined with classical runtime results, additive landscapes give $O(n \log k)$ re-adaptation (Proposition 2.2).
- C3 Method.** *Evolution-in-the-loop* meta-training: an inner EA solves sampled tasks from rewards; the encoder is updated by gradients evaluated *at the genomes the EA found*, plus an epistasis regulariser (Section 4).
- C4 Evaluation.** A staged plan, from a synthetic task family with known latent factors to reward-only adaptation on real images, with baselines that isolate each claim (Section 6).

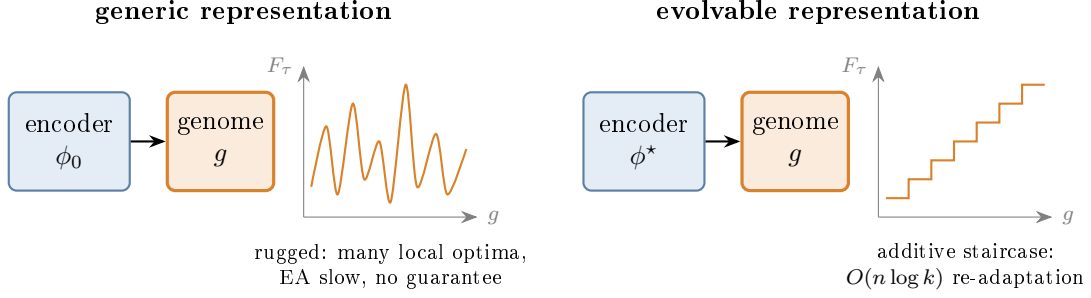


Figure 1: Same task, different landscape. The fitness landscape an EA sees over the genome is determined jointly by the task and the frozen representation. This proposal trains the representation so that the landscape is close to additive, where EA runtime is fast and provable. (Schematic.)

2 Setting and notation

2.1 Task family, encoder, genome

Symbol	Meaning
$\tau \sim P(\mathcal{T})$	a task from a task family; $\mathcal{D}_\tau$ its distribution over (x, y) , $y \in \{-1, +1\}$
$f_\phi : \mathcal{X} \rightarrow \{0, 1\}^n$	encoder with parameters ϕ producing n binary features (the slow system)
$g \in \mathcal{G} = \{-1, 0, 1\}^n$	ternary genome: each feature votes for, ignores, or votes against (the fast system)
$m_g(x) = \langle g, f_\phi(x) \rangle - \theta$	read-out pre-activation with fixed threshold $\theta \notin \mathbb{Z}$ (no ties)
$\hat{y}_g(x) = \text{sign } m_g(x)$	prediction; $r_g(x, y) = \mathbf{1}[y m_g(x) > 0]$ its correctness
$F_\tau(g; \phi) = \mathbb{E}_{\mathcal{D}_\tau}[r_g(x, y)]$	fitness: expected reward (accuracy) of genome g on task τ
$g_\tau^*(\phi)$	an optimal genome for τ under ϕ
$\mathcal{A}$	an evolutionary algorithm that can only query (noisy estimates of) F_τ
$T_{\mathcal{A}}(g_0 \rightarrow \tau; \phi)$	number of fitness evaluations $\mathcal{A}$ needs, started at g_0 , to reach $(1 - \epsilon) \max_g F_\tau$
$\text{Ham}(g, g')$	number of positions in which two genomes differ

The read-out is deliberately the simplest non-trivial one, a single ternary threshold unit (a perceptron with weights in $\{-1, 0, 1\}$), so that the landscape is a pseudo-Boolean-like function on a finite alphabet, the object that EA runtime theory studies. Multi-class and multi-output read-outs are obtained by stacking units with a fixed code (as in error-correcting output codes); every statement below then applies per unit.

Information available.

- **Meta-training (offline, slow).** Labelled samples from a family of training tasks, as in standard meta-learning. The encoder may be trained with gradients.
- **Deployment (online, fast).** For a new task, only the scalar batch reward $\hat{F}_\tau(g) = \frac{1}{B} \sum_b r_g(x_b, y_b)$ of each queried genome is observed. No labels, no gradients. This is the regime where an EA is the natural learner, rather than a worse replacement for gradient descent.

2.2 Evolvability of a representation

Let $P(\tau, \tau')$ be a distribution over pairs of *related* tasks (a task and its successor in a changing environment).

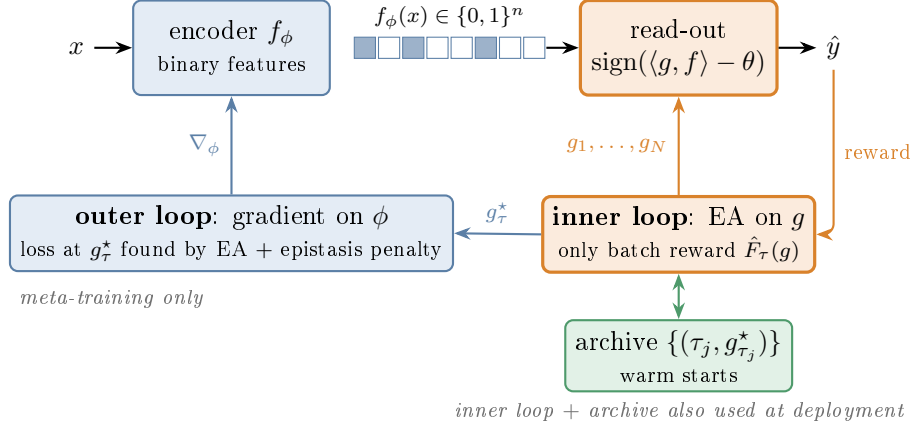


Figure 2: Evolution in the loop. A slow encoder produces binary features; a tiny ternary genome votes over them. During meta-training, an inner EA solves each sampled task from rewards only, and the encoder is updated by gradients evaluated at the genomes the EA found. At deployment, only the inner loop and the archive remain: adaptation is reward-only, parallel and leaves the encoder untouched.

Definition 2.1 (Capacity and evolvability). For an encoder ϕ and an EA $\mathcal{A}$,

$$\text{Cap}(\phi) = \mathbb{E}_{\tau} \left[\max_{g \in \mathcal{G}} F_{\tau}(g; \phi) \right], \quad (1)$$

$$\text{Evo}_{\mathcal{A}}(\phi) = \mathbb{E}_{(\tau, \tau') \sim P} \left[T_{\mathcal{A}}(g_{\tau}^*(\phi) \rightarrow \tau'; \phi) \right]. \quad (2)$$

Cap is what the representation *can* express with a ternary vote; Evo is how many reward evaluations it costs to *move* between related tasks. We seek representations with high capacity and low Evo.

The two quantities are in tension. The identity representation of raw pixels has low capacity for a single threshold unit. A representation with one feature per task ("grandmother cells") makes every task trivially solvable but relies on those tasks having been anticipated. The interesting regime lies in between: features that are *reusable building blocks* of many tasks.

2.3 Two knobs: ruggedness and task distance

For single-position changes δ_i (setting g_i to another value) define the first and second differences

$$\Delta_i F(g) = F(g \oplus \delta_i) - F(g), \quad \Delta_{ij}^2 F(g) = F(g \oplus \delta_i \oplus \delta_j) - F(g \oplus \delta_i) - F(g \oplus \delta_j) + F(g). \quad (3)$$

F is *additive* if all $\Delta_{ij}^2 F \equiv 0$. We measure ruggedness by the *relative epistasis*

$$\varepsilon_{\tau}(\phi) = \frac{\mathbb{E}_g \mathbb{E}_{i \neq j} |\Delta_{ij}^2 F_{\tau}(g; \phi)|}{\mathbb{E}_g \mathbb{E}_i |\Delta_i F_{\tau}(g; \phi)|}, \quad (4)$$

with g drawn along typical EA trajectories. The second knob is the *task distance* $k_{\tau\tau'}(\phi) = \text{Ham}(g_{\tau}^*(\phi), g_{\tau'}^*(\phi))$: how many votes must change when the task changes.

Proposition 2.2 (Re-adaptation on additive landscapes). *Let F be OneMax-like on $\{-1, 0, 1\}^n$, i.e. $F(g) = -|\{i : g_i \neq g_i^*\}|$. The $(1+1)$ EA with per-position mutation probability $1/n$ (to a uniformly random other value), started at Hamming distance k from g^* , needs in expectation at most*

$$\mathbb{E}[T_k] \leq 2enH_k = O(n \log k)$$

evaluations, where $H_k = \sum_{i=1}^k 1/i$.

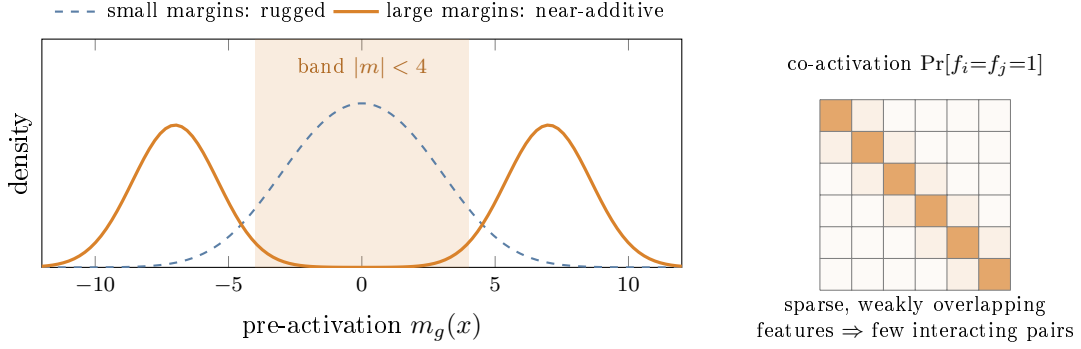


Figure 3: The two feature-level sources of epistasis (Lemma 3.1), schematic. Left: only examples whose pre-activation falls into the band $|m| < 4$ can make two vote changes interact; large margins push mass out of the band. Right: only co-active feature pairs can interact; sparse, weakly overlapping features have few such pairs.

Proof. At distance i , mutating exactly one of the i wrong positions to its correct value, and nothing else, has probability at least $i \cdot \frac{1}{n} \cdot \frac{1}{2} \cdot (1 - \frac{1}{n})^{n-1} \geq \frac{i}{2en}$. Elitism never increases the distance, so the expected waiting time at distance i is at most $2en/i$; summing over $i = 1, \dots, k$ gives the claim. $\square$

Remark 2.3. Under exact additivity, Evo therefore scales like $n \mathbb{E}[\log k_{\tau\tau'}]$. A representation should thus make the landscape (nearly) additive, keep n moderate, and make *related tasks differ in few votes*. The last point echoes the biological finding that environments whose goals vary in a modular way produce modular, evolvable networks. For weighted additive functions over finite alphabets, $O(n \log n)$ bounds from arbitrary starting points are known; distance- k bounds for weighted additive functions are an open question we intend to study (Section 5).

3 Where epistasis comes from: a feature-level bound

The landscape of a threshold read-out is never exactly additive, because correctness is a step function of a sum. The following lemma says precisely *where* the non-additivity comes from, and it only involves quantities that the encoder controls.

Lemma 3.1 (Epistasis is co-activation near the threshold). *Let $f_\phi(x) \in \{0, 1\}^n$, $g \in \{-1, 0, 1\}^n$, $m_g(x) = \langle g, f_\phi(x) \rangle - \theta$ and $F(g) = \mathbb{E}[\mathbf{1}[y m_g(x) > 0]]$. Then for all g and all single-position changes δ_i, δ_j with $i \neq j$,*

$$|\Delta_{ij}^2 F(g)| \leq 2 \Pr_x [f_i(x) = f_j(x) = 1 \text{ and } |m_g(x)| < 4]. \quad (5)$$

Proof. Fix (x, y) and write $a = (g'_i - g_i)f_i(x)$, $b = (g'_j - g_j)f_j(x)$ for the changes of the pre-activation caused by δ_i and δ_j ; note $|a|, |b| \leq 2$. The per-example second difference is $D = \rho(m + a + b) - \rho(m + a) - \rho(m + b) + \rho(m)$ with $\rho(t) = \mathbf{1}[yt > 0]$. If $f_i(x) = 0$ then $a = 0$ and $D = \rho(m + b) - \rho(m) - \rho(m + b) + \rho(m) = 0$; similarly if $f_j(x) = 0$. If $|m| > 4$, all four arguments $m, m + a, m + b, m + a + b$ lie on the same side of 0 (since $|a + b| \leq 4$, and $|m| = 4$ is impossible because $\langle g, f \rangle \in \mathbb{Z}$ and $\theta \notin \mathbb{Z}$), so ρ is constant and $D = 0$. In all remaining cases $|D| \leq 2$, since D is a signed sum of four indicators with two positive and two negative signs. Taking expectations gives the bound. $\square$

Remark 3.2 (Consequences). Summing over pairs, the total epistasis at g satisfies $\sum_{i \neq j} |\Delta_{ij}^2 F(g)| \leq 2 \mathbb{E}[c(x)(c(x) - 1) \mathbf{1}[|m_g(x)| < 4]]$, where $c(x) = \|f_\phi(x)\|_1$ is the number of active features. Two regimes make the landscape near-additive:

- **sparse regime**: few features active per input ($c(x)$ small), so few pairs can interact;
- **margin regime**: most inputs far from the threshold, which requires *many agreeing votes*, i.e. redundancy.

The two pull in opposite directions through $c(x)$. Which one a trained encoder chooses, and whether a mixture (sparse features with redundant copies) is optimal, is a concrete question the experiments can answer.

Remark 3.3 (Why a threshold read-out and not a linear one). If per-example labels were available, a linear surrogate $\sum_b y_b \langle g, f(x_b) \rangle$ would be maximised in closed form, and no EA would be needed. The EA earns its place exactly because deployment provides only scalar rewards, and the lemma shows that the non-linearity of the thresholded reward is *local* (confined to the band and co-active pairs), so it can be controlled.

4 Method: evolution-in-the-loop meta-training

4.1 Objective

We train ϕ on a family of training tasks to maximise capacity while minimising the two drivers of Evo:

$$\min_{\phi} \mathbb{E}_{\tau} [\mathcal{L}_{\tau}(\phi; g_{\tau}^*)] + \lambda_{\text{co}} \mathcal{R}_{\text{co}}(\phi) + \lambda_{\text{band}} \mathbb{E}_{\tau} \mathcal{R}_{\text{band}}(\phi; g_{\tau}^*), \quad (6)$$

where g_{τ}^* is the genome returned by the inner EA (treated as a constant), and

$$\mathcal{L}_{\tau}(\phi; g) = \mathbb{E}_{\mathcal{D}_{\tau}} [\max(0, \gamma - y \tilde{m}_g(x))], \quad \tilde{m}_g(x) = \langle g, \tilde{f}_{\phi}(x) \rangle - \theta, \quad (7)$$

$$\mathcal{R}_{\text{co}}(\phi) = \sum_{i \neq j} \mathbb{E}_x [\tilde{f}_i(x) \tilde{f}_j(x)], \quad \mathcal{R}_{\text{band}}(\phi; g) = \mathbb{E}_x [\tilde{c}(x)^2 \kappa(\tilde{m}_g(x))]. \quad (8)$$

Here $\tilde{f}_{\phi} = \sigma(h_{\phi}/T)$ are soft features (hard thresholding with a straight-through estimator at the end of training), $\tilde{c} = \|\tilde{f}\|_1$, and $\kappa(t) = \sigma((4 - |t|)/T)$ is a smooth indicator of the epistatic band. The hinge loss with margin $\gamma \geq 4$ pushes examples out of the band on the task the genome solves; $\mathcal{R}_{\text{co}}$ targets the sparse regime, and $\mathcal{R}_{\text{band}}$ is the smooth version of the bound in Remark 3.2.

4.2 Algorithm

Algorithm 1: Evolution-in-the-loop meta-training

Input: task family $P(\mathcal{T})$; encoder f_{ϕ} ; EA $\mathcal{A}$ with inner budget T_{in} ; archive $\mathcal{M} = \emptyset$

```

1 for outer step  $s = 1, 2, \dots$  do
2   sample tasks  $\tau_1, \dots, \tau_M \sim P(\mathcal{T})$  (including successors  $\tau'$  of earlier tasks);
3   for  $\tau \in \{\tau_1, \dots, \tau_M\}$  do                                     // parallel over tasks
4      $g_0 \leftarrow$  best archived genome for a related task (or 0);
5      $g_{\tau}^* \leftarrow \mathcal{A}(g_0; \hat{F}_{\tau})$  with budget  $T_{\text{in}}$ , using only batch rewards on hard features;
6     record  $T_{\mathcal{A}}$  and  $\text{Ham}(g_0, g_{\tau}^*)$ ; store  $(\tau, g_{\tau}^*)$  in  $\mathcal{M}$ ;
7    $\phi \leftarrow \phi - \eta \nabla_{\phi} \left[ \frac{1}{M} \sum_{\tau} \mathcal{L}_{\tau}(\phi; g_{\tau}^*) + \lambda_{\text{co}} \mathcal{R}_{\text{co}} + \lambda_{\text{band}} \frac{1}{M} \sum_{\tau} \mathcal{R}_{\text{band}}(\phi; g_{\tau}^*) \right];$ 

```

The update is first-order: no gradient flows through the EA, exactly as in first-order MAML or Reptile. What makes it different from standard multi-task training is *which* read-out the gradient is evaluated at: not a jointly trained linear head, but the discrete genome a black-box EA *actually finds* within a small budget. The encoder is therefore pressured to make EA-reachable solutions good, which is a form of *learning shaped by evolution*. It is the reverse direction of the Baldwin effect, where learning shapes evolution.

4.3 Why this is cheap

The inner loop acts on cached binary features: evaluating N genomes on a batch of B inputs is one $B \times n$ by $n \times N$ integer matrix product. With $n = 64$, $B = 512$ and $N = 256$ this is $\approx 8 \cdot 10^6$ operations, i.e. microseconds to milliseconds on a CPU. The encoder is small in the synthetic stages and a fixed pretrained backbone plus a small trainable head in the real-image stage. The whole programme is sized for a CPU cluster (Euler) and a single T4 GPU.

5 Research questions and hypotheses

RQ1 Measurability. Can $\varepsilon_\tau(\phi)$ and $k_{\tau\tau'}(\phi)$ be estimated reliably, and do they predict EA adaptation cost across representations?

H1: across encoders, $\log T_A$ is well predicted by ε and $\log k$.

RQ2 Shapeability. Does evolution-in-the-loop training reduce ε and k compared to generic representations of equal capacity?

H2: at matched Cap, the shaped encoder has lower ε and lower median k than supervised multi-task, autoencoder and disentanglement-oriented encoders.

RQ3 Payoff. Does lower ε translate into faster reward-only adaptation than strong gradient-free and policy-gradient baselines?

H3: evaluations to reach 95% of the achievable reward drop by a large factor, and the drop grows with how related consecutive tasks are.

RQ4 Theory. How far do runtime guarantees extend beyond exact additivity?

Targets: distance- k bounds for weighted additive functions on finite alphabets; bounds for the $(1+1)$ EA in terms of ε or of the co-activation statistic of Lemma 3.1; which of the sparse and margin regimes is preferable.

6 Experimental plan

All stages share the protocol: meta-train on a family of tasks, then evaluate reward-only adaptation on *held-out* tasks and task transitions. Every EA-based arm is run on every representation, so that the representation effect is isolated from the optimiser effect.

6.1 Stage 1 — synthetic task family with known ground truth (CPU)

- **Generator.** Latent binary factors $s \in \{0, 1\}^D$ ($D = 16$), i.i.d. Bernoulli($\frac{1}{2}$). Observations $x = M_2 \tanh(M_1 s + \xi) \in \mathbb{R}^{64}$ with fixed random matrices M_1, M_2 and small noise ξ : an unknown non-linear entanglement of the factors.
- **Tasks.** $y_\tau(s) = \text{sign}(\sum_{j \in S_\tau} a_j(2s_j - 1))$ with $|S_\tau| = 3$ and $a_j \in \{\pm 1\}$. *Related* successors change one element of S_τ or one sign, so the ground-truth change is small and known.
- **Why it is useful.** The ideal representation is known (the factors themselves, possibly with redundant copies), so capacity, the achievable k , and the effect of shaping can be checked against ground truth.
- **Encoder.** Two-layer MLP to $n \in \{32, 64, 128\}$ binary units.

6.2 Stage 2 — images with known factors (CPU / T4)

dSprites or Shapes3D, whose generating factors (shape, scale, position, colour, ...) are labelled. Tasks are sparse threshold rules over binarised factors, exactly as in Stage 1. Encoder: a small

CNN. This stage tests whether shaping survives real perceptual input, and relates evolvability to disentanglement, which is measurable here.

6.3 Stage 3 — reward-only adaptation on natural images (T4)

Frozen pretrained backbone (e.g. ResNet-18 or a small ViT) with cached features. The *trainable* encoder is a small head mapping these features to n binary units. Tasks are binary rules over CIFAR-100 classes, for example “is it one of these three superclasses”. Successor tasks change one class group. Deployment sees only batch accuracy.

6.4 Stage 4 — scaling path (with large compute)

Stages 1–3 answer the scientific question on hardware available now (CPU cluster, one T4). With large compute, the same question transfers directly to language models, where reward-only adaptation is currently a central topic.

- **Slow system = a mixture-of-experts LLM.** The “features” are the router inputs and the experts of the last L' MoE layers (e.g. OLMoE, 64 experts per layer). The **genome** is a ternary or small-integer vector of per-expert routing biases $b_\ell \in \{-1, 0, 1\}^K$ (“force-exclude / neutral / prefer”), i.e. exactly the read-out of Section 2, one level up.
- **Deployment:** reward-only domain adaptation, where the reward is the accuracy or verifier score on a batch of prompts from a new domain. The prompt prefix up to layer $L - L'$ is computed once and shared by the whole population, so evaluating N genomes costs roughly one forward pass plus N short suffixes.
- **Shaping:** evolution-in-the-loop fine-tuning of the last-layer experts and routers. The inner EA finds routing genomes for training domains; the outer gradient updates experts and routers at those genomes, with the co-activation regulariser applied to expert co-selection (experts that are chosen together interact).
- **Measurements** are unchanged: ε and k over routing genomes, evaluations to adapt, and comparisons with test-time routing methods (C3PO), policy-gradient fine-tuning (GRPO on LoRA) and ES fine-tuning of full weights (EGGROLL-style).
- **Analogous instance:** a LoRA library in place of experts, with the genome as composition coefficients.

The small-scale stages are therefore not a toy detour. They build and validate the instruments (ε , k , evaluations-to-adapt, the lemma-derived regulariser) that the large-scale stage relies on.

6.5 Arms

	arm	what it isolates
<i>Representations (each combined with the same EAs)</i>		
R0	random binary projection	floor
R1	supervised multi-task encoder (jointly trained linear heads, then binarised)	standard pretraining
R2	autoencoder / β -VAE, binarised	reconstruction and disentanglement objectives
R3	evolution-in-the-loop, $\lambda_{\text{co}} = \lambda_{\text{band}} = 0$	effect of training <i>at EA-found genomes</i> alone
R4	evolution-in-the-loop, full objective	proposed
R5	R4 with randomised co-activation targets	control: regularisation vs. structure
<i>Adaptation algorithms at deployment (reward-only)</i>		
A1	(1+1) EA, ternary mutation	matches the theory
A2	PBIL / cGA with population	parallel EA
A3	CMA-ES on a continuous head over the same features	continuous gradient-free baseline
A4	REINFORCE / ES on a continuous head	policy-gradient baseline
A5	ES-MAML: meta-learned initialisation, ES adaptation	meta-learning for ES
A6	ANIL-style head fine-tuning <i>with labels</i>	reference upper bound, not reward-only

6.6 Metrics

- reward evaluations to reach 95% of the task’s achievable reward, for fresh tasks and for related successors;
- ε_τ , the distribution of $k_{\tau\tau'}$, and the co-activation and band statistics of Lemma 3.1;
- capacity Cap (achievable reward with a ternary vote) and final reward;
- fit of adaptation time against $n \log k$ (slope and residuals) to test how far the additive theory carries;
- stages 2–3: standard disentanglement scores, to relate evolvability to disentanglement.

Each configuration is run with at least 10 seeds. Controls get independent seeds, and every baseline’s step size or learning rate is tuned. This follows lessons from earlier work in this project, where two conclusions were overturned by controls that could not vary.

6.7 Compute and timeline

weeks	stage	deliverable
1–2	reading, generator, EA library	Stage-1 generator; (1+1) EA/PBIL on cached binary features; ε and k estimators
3–5	Stage 1 (Euler CPU)	RQ1 and RQ2 on ground truth; first go/no-go
6–8	Stage 2 (CPU/T4)	shaping on dSprites/Shapes3D; link to disentanglement
9–11	Stage 3 (T4)	reward-only adaptation vs. A3–A6
12–14	theory and write-up	RQ4 results; report
later	Stage 4 (large compute)	MoE routing genomes and LoRA composition at LLM scale (Section 6.4)

Go/no-go after Stage 1. If, at matched capacity, R4 does not reduce ε *and* adaptation evaluations clearly below R1/R2, then shaping for evolvability does not work in the cleanest possible setting. The project then pivots to a measurement study (RQ1 and RQ4): which existing representations are evolvable, and why.

7 Related work

Evolvability. In evolutionary biology, evolvability is the capacity of a genotype–phenotype map to produce adaptive variation [21]. Modularly varying goals make networks modular [13], and connection costs make them modular and more evolvable [2]. In evolutionary computation, Evolvability ES directly optimises the behavioural diversity of a policy’s offspring [9]. We transfer the concept to learned representations and make it quantitative through the landscape an EA sees over a late genome, with runtime-theoretic consequences.

Meta-learning. MAML learns an initialisation from which gradient steps adapt fast [8]. ANIL shows that most of MAML’s benefit is feature reuse with a rapidly adapted head [17], the gradient counterpart of our slow encoder with a fast genome. ES-MAML replaces the inner gradient by ES [19] and has been used for rapidly adaptable legged robots [20]. Meta-learning through the Baldwin effect evolves initial parameters and hyperparameters for within-lifetime learning [7]. All of these shape an *initialisation* for a continuous inner learner; we shape the *landscape* of a discrete, reward-only inner learner and target provable runtime.

Fitness landscapes and runtime theory. NK landscapes tune ruggedness through the degree of epistatic interaction [14], and fitness–distance correlation predicts EA difficulty [12]. The (1+1) EA optimises linear pseudo-Boolean functions in $\Theta(n \log n)$ [4, 22], also over finite alphabets [3]; dynamic and re-optimisation settings have been studied for OneMax variants and linear functions [15, 18]. We ask which learned representations make real adaptation problems fall into these tractable classes.

Representations and disentanglement. Disentangled representations aim to separate generative factors; their benefits and identifiability are debated [16]. Variance–covariance regularisation decorrelates features [1]. Our co-activation regulariser is related, but it is *derived* from an epistasis bound rather than posited, and it is evaluated by a downstream search cost.

Late evolved components and discrete masks. Evolving a small controller on top of learned models [10], gradient-free composition of adapters [11], and selecting paths or binary masks over frozen networks for continual learning [6, 23] all place a small search problem on top of a fixed representation. None of them shape the representation for the search. Scaling ES to large models [5] is the complementary direction.

8 Expected contributions and outlook

1. A definition of evolvability for learned representations that is measurable and connects EA runtime theory to representation learning.
2. A feature-level epistasis bound for threshold read-outs, and theory on re-adaptation beyond exact additivity.
3. Evolution-in-the-loop meta-training, with evidence on whether it yields representations on which reward-only evolutionary adaptation is fast.
4. A benchmark suite of task families with known ground truth for studying evolutionary adaptation on learned features.

Outlook. If representations can be made evolvable, the slow/fast split becomes a principled design for systems that keep adapting after deployment. The encoder is updated rarely and by gradients; the genomes adapt continuously from rewards, in parallel and without forgetting, stored in an archive that doubles as memory. The same question then applies to richer genomes: routing in mixtures of experts, or composition of adapter libraries.

References

- [1] A. Bardes, J. Ponce, Y. LeCun. VICReg: variance-invariance-covariance regularization for self-supervised learning. ICLR 2022. arXiv:2105.04906.
- [2] J. Clune, J.-B. Mouret, H. Lipson. The evolutionary origins of modularity. Proc. R. Soc. B 280, 2013. arXiv:1207.2743.
- [3] B. Doerr, S. Pohl. Run-time analysis of the (1+1) evolutionary algorithm optimizing linear functions over a finite alphabet. GECCO 2012.
- [4] S. Droste, T. Jansen, I. Wegener. On the analysis of the (1+1) evolutionary algorithm. Theoretical Computer Science 276, 2002.
- [5] Evolution strategies at the hyperscale (EGGROLL). arXiv:2511.16652, 2025.
- [6] C. Fernando et al. PathNet: evolution channels gradient descent in super neural networks. arXiv:1701.08734, 2017.
- [7] C. Fernando et al. Meta-learning by the Baldwin effect. GECCO 2018 companion. arXiv:1806.07917.
- [8] C. Finn, P. Abbeel, S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. ICML 2017. arXiv:1703.03400.
- [9] A. Gajewski, J. Clune, K. O. Stanley, J. Lehman. Evolvability ES: scalable and direct optimization of evolvability. GECCO 2019. arXiv:1907.06077.
- [10] D. Ha, J. Schmidhuber. Recurrent world models facilitate policy evolution. NeurIPS 2018. arXiv:1803.10122.
- [11] C. Huang et al. LoraHub: efficient cross-task generalization via dynamic LoRA composition. COLM 2024. arXiv:2307.13269.
- [12] T. Jones, S. Forrest. Fitness distance correlation as a measure of problem difficulty for genetic algorithms. ICGA 1995.
- [13] N. Kashtan, U. Alon. Spontaneous evolution of modularity and network motifs. PNAS 102(39), 2005.
- [14] S. Kauffman, S. Levin. Towards a general theory of adaptive walks on rugged landscapes. Journal of Theoretical Biology 128, 1987.
- [15] T. Kötzing, A. Lissovoi, C. Witt. (1+1) EA on generalized dynamic OneMax. FOGA 2015.

- [16] F. Locatello et al. Challenging common assumptions in the unsupervised learning of disentangled representations. ICML 2019. arXiv:1811.12359.
- [17] A. Raghu, M. Raghu, S. Bengio, O. Vinyals. Rapid learning or feature reuse? Towards understanding the effectiveness of MAML. ICLR 2020. arXiv:1909.09157.
- [18] F. Shi, M. Schirneck, T. Friedrich, T. Kötzing, F. Neumann. Reoptimization time analysis of evolutionary algorithms on linear functions under dynamic uniform constraints. *Algorithmica* 81, 2019.
- [19] X. Song et al. ES-MAML: simple Hessian-free meta learning. ICLR 2020. arXiv:1910.01215.
- [20] X. Song et al. Rapidly adaptable legged robots via evolutionary meta-learning. IROS 2020. arXiv:2003.01239.
- [21] G. P. Wagner, L. Altenberg. Complex adaptations and the evolution of evolvability. *Evolution* 50(3), 1996.
- [22] C. Witt. Tight bounds on the optimization time of a randomized search heuristic on linear functions. *Combinatorics, Probability and Computing* 22, 2013.
- [23] M. Wortsman et al. Supermasks in superposition. NeurIPS 2020. arXiv:2006.14769.